Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

2026
Vladimír Jančár

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

Study Programme: Applied Computer Science
Field of Study: Computer Science
Department: Department of Applied Informatics
Supervisor: Mgr. Ján Pastorek

Bratislava, 2026
Vladimír Jančár

16681876

Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

THESIS ASSIGNMENT

Name and Surname:	Vladimír Jančár
Study programme:	Applied Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study:	Computer Science
Type of Thesis:	Bachelor´s thesis
Language of Thesis:	English
Secondary language:	Slovak

Title: Machine learning for generation of graph of given degree and girth.

Annotation: Generating graphs with prescribed structural properties is a key task in combinatorial optimization, network design, and bioinformatics. Traditional algorithmic approaches (e.g., configuration model) often fail to find graphs with high girth and specific degree because the space of possible solutions is vast and the constraints are nontrivial.

Aim: The goal is to investigate potential applications of machine learning to problems in algebraic graph theory. The student will have the opportunity to engage with the state-of-art research in machine learning and algebraic graph theory and contribute to the field by generating new record graphs and training neural networks to predict algebraic properties.

Literature: Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., & Grohe, M. (2019). Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks. _Proceedings of the AAAI Conference on Artificial Intelligence_, _33_(01), 4602–4609. [https://doi.org/10/ggfn97] (https://doi.org/10/ggfn97)
Paolino, R., Maskey, S., Welke, P., & Kutyniok, G. (2024). _Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning_.
Wu, L., Cui, P., Pei, J., & Zhao, L. (Eds.). (2022). _Graph Neural Networks: Foundations, Frontiers, and Applications_. Springer Nature Singapore. [https://doi.org/10.1007/978-981-16-6054-2] (https://doi.org/10.1007/978-981-16-6054-2)

Supervisor:	Mgr. Ján Pastorek
Department:	FMFI.KAI - Department of Applied Informatics
Head of department:	doc. RNDr. Tatiana Jajcayová, PhD.
Assigned:	25.02.2025

Approved:

Guarantor of Study Programme

.....
Student

.....
Supervisor

Acknowledgments:

[Acknowledgments to be written near the end of the thesis. Mention the supervisor, people who helped with experiments or computing resources, and any personal thanks that should appear in the submitted version.]

Abstract

[One- to two-sentence framing of the overall goal: machine-learning methods for generating graphs of prescribed degree and girth, i.e. (k, g)-graphs and cage candidates.]

[Paragraph on the preparatory subtasks: which graph-property prediction tasks were studied (degree prediction, minimum-cycle / girth-related prediction), which GNN architectures were compared, and what the headline finding of these preparatory experiments is.]

[Paragraph on the main construction approaches that were tried: direct reinforcement-learning graph editing, voltage graph lifts with non-learned search, and voltage graph lifts with learned policies. Briefly state what each approach contributes.]

[Closing sentence stating the main experimental conclusion and what it implies about the role of GNNs in this setting. To be finalized once the final wording of the main claim is fixed.]

Keywords: [keywords to be finalized; current working set: graph neural networks, algebraic graph theory, cage problem, (k, g)-graphs, voltage graph lifts, heuristic search]

Contents

List of Figures

List of Tables

1 Introduction

This thesis concerns machine-learning methods for generating graphs of prescribed degree and girth. In graph-theoretic language, the central objects are (k, g)-graphs: graphs in which every vertex has degree k and every cycle has length at least g. The degree fixes the local shape of the graph, while the girth says that short loops are forbidden. The classical cage problem asks for such graphs with the smallest possible number of vertices [1].

Practical applications of (k, g)-graphs come from situations where a system should have uniform local connectivity but should avoid short feedback loops. In error-correcting codes, sparse bipartite graphs describe which symbols participate in which parity checks. Decoding algorithms pass messages along this graph. If the graph has many short cycles, the same information can return to a vertex too quickly and the messages become correlated; larger girth keeps the local neighborhood tree-like for more decoding rounds [2]. Small high-girth regular graphs are therefore useful as compact templates for reliable communication and storage systems.

The same design tension appears in communication networks and supercomputer interconnects. A switch or processor has only a limited number of physical ports, so the degree is constrained by hardware. At the same time, one wants many short routes between machines without local congestion caused by tightly looping connections. Regular high-girth and near-Moore graphs provide blueprints for such networks; the Slim Fly topology, for example, is built from degree-diameter and near-Moore graph ideas and was proposed as a low-cost, low-latency interconnect [3]. In this sense, constructing smaller or better (k, g)-graphs is not only an abstract extremal problem: it enlarges the catalogue of graph topologies available for codes, networks, and other structured systems.

The mathematical task is simple to state but difficult to solve: for fixed k and g, construct a small graph satisfying both constraints. Known approaches use algebraic constructions, voltage graph lifts, exhaustive search, heuristic search, and local modification techniques [4]. This thesis asks whether graph neural networks can help in this setting. The expected role of machine learning is not to replace exact graph-theoretic checks, but to guide a search toward promising choices.

The thesis is organized around the following research questions:

- Which graph neural network architectures can learn the local and cycle-related properties that appear in the definition of a (k, g)-graph?
- Can reinforcement learning construct small (k, g)-graphs by editing edges?
- Does an algebraic representation, in particular voltage graph lifts, give a more useful search space?
- Does learning improve this constrained search, or do non-learned methods remain stronger?

The experiments are organized around a sequence of increasingly constrained formulations. Prediction tasks test whether the models learn local and cycle-related graph information. Direct reinforcement learning then treats construction as an interaction problem, where the model chooses graph edits and receives rewards from the resulting state. Voltage graph lifts provide the most structured formulation: regularity is guaranteed by the base graph, and the search focuses on voltage assignments that avoid short cycles.

2 Background and Definitions

Only the notation needed for the experiments is stated here. More specialized constructions, such as voltage graph lifts, are introduced later together with the methods that use them.

2.1 Graphs

A *graph* is a pair $G = (V, E)$, where V is a finite set of vertices and E is a set of edges. In the main construction setting, graphs are simple and undirected unless stated otherwise.

The *degree* of a vertex $v \in V$, denoted $\deg(v)$, is the number of edges incident with v. The *open neighborhood* of v, denoted $N(v)$, is the set of vertices adjacent to v.

A graph is k-regular if every vertex has degree k.

A *walk* is a sequence of vertices in which consecutive vertices are adjacent.

A *path* is a walk with no repeated vertices.

A *cycle* is a closed path of length at least 3.

The *girth* of a graph, denoted $\mathrm{girth}(G)$, is the length of its shortest cycle. If a graph has no cycles, its girth is treated as infinite.

A vertex v is *active* if $\deg(v) > 0$. The *active subgraph* of G is the subgraph induced by the active vertices, i.e. G with all isolated vertices removed. This is convenient when a construction algorithm carries a large fixed vertex pool but uses only the vertices it has already drawn into edges.

2.2 (k, g)-Graphs and Cages

A (k, g)-graph is a k-regular graph whose girth is at least g. A (k, g)-cage is a smallest k-regular graph of girth g; its order is commonly denoted $n(k, g)$. In search algorithms it is convenient to test the condition $\mathrm{girth}(G) \geq g$, because this accepts every valid candidate for a target lower bound on girth. To compare two constructions, one must also record the number of vertices and the actual girth of the graph found.

The Moore bound gives a general lower bound on the order of a (k, g)-graph. It is useful both as a theoretical benchmark and as a practical target size for construction algorithms.

For $k \geq 2$ and $g \geq 3$, the Moore bound is defined as follows. If g is odd, then

$$M(k, g) = 1 + k \sum_{i=0}^{\frac{g-3}{2}} (k-1)^i.$$

If g is even, then

$$M(k, g) = 2 \sum_{i=0}^{\frac{g-2}{2}} (k-1)^i.$$

A graph meeting this bound is called a Moore graph. Moore graphs are rare. For most parameter pairs, the practical problem is therefore to close the gap between the Moore lower bound and the best published upper bounds.

2.3 Graph Neural Networks

Ordinary feed-forward neural networks expect fixed-size vectors. Graphs do not have a fixed number of vertices, and their vertices do not have a canonical ordering. Graph neural networks address this by applying the same local update rule at every vertex. This makes the model independent of the input graph size and equivariant to relabeling of vertices.

Message-passing graph neural networks process graph-structured data by iteratively updating vertex representations through layers [5]. At layer t, each vertex v has a hidden representation h_v^t. The layer aggregates messages from the neighborhood $N(v)$ and then updates the representation:

$$m_v^t = \mathrm{AGG}\big(\{h_u^{t-1} : u \in N(v)\}\big)$$

$$h_v^t = \mathrm{UPD}\big(h_v^{t-1}, m_v^t\big).$$

Several parameters determine what such a model can express. The *number of layers* controls how far information can travel: after one layer, a vertex representation depends on its immediate neighbors; after two layers, it can depend on vertices at distance two; and so on. The *hidden dimension* controls how much information can be stored in each vertex representation. The *aggregation rule* determines which neighborhood statistics are easy to preserve; for example, normalized aggregation can make raw counting harder, while sum-like aggregation keeps count information more directly. Edge features, positional encodings, and graph-level pooling become important when the task depends on edge labels or on the position of a vertex inside the whole graph. This is why local quantities such as degree are easier for standard GNNs than global quantities such as girth.

Grohe's survey on the logic of graph neural networks provides the theoretical framing for this limitation [6]. Standard message-passing GNNs are closely related to color refinement and the one-dimensional Weisfeiler–Leman algorithm. This con-

nection is important for the thesis because it explains why some graph properties are naturally accessible to ordinary GNNs while other global or symmetry-sensitive properties may require stronger architectures, additional features, or a different formulation of the search problem.

The experiments use several GNN architectures:

- *GCN*, a stable baseline based on degree-normalized neighbor aggregation [7].
- *GraphSAGE*, an inductive architecture based on learned aggregation [8].
- *GIN*, a sum-aggregation architecture with expressivity related to the Weisfeiler–Leman test [9].
- *GINE*, an edge-aware variant of GIN used when edge attributes such as voltage labels are part of the input [10].
- *GPS*, a graph transformer architecture combining local message passing with global attention [11]. GPS models often benefit from structural or positional encodings, such as random-walk-based encodings, because attention by itself does not identify a vertex's structural role in the graph.
- *Loopy GNN*, a cycle-aware architecture relevant to minimum-cycle and girth-related tasks [12].

3 Related Work

The work builds on two lines of research: graph-theoretic construction of small regular graphs, and learned heuristics for difficult discrete search problems. The relevant background is not every known cage construction, but the recurring pattern behind successful methods: restrict the search space mathematically, then use computation to explore the remaining choices.

3.1 Classical and Computational Cage Construction

The cage problem has been studied through algebraic constructions, finite geometries, Cayley graphs, voltage graph lifts, exhaustive generation, local search, and excision. The Dynamic Cage Survey provides the broad background and record-holder context [1].

The most relevant lesson from classical construction is that successful methods rarely search over arbitrary graphs. They usually impose structure first and search only inside the remaining degrees of freedom. Algebraic families illustrate this directly. Incidence graphs of finite projective planes and generalized quadrangles yield $(q+1)$-regular graphs of girth 6 and 8 for prime powers q; generalized hexagons give analogous constructions for girth 12. Cayley graphs on suitable groups produce highly symmetric candidates, and Ramanujan graphs [13] provide explicit algebraic families with optimal spectral properties and large girth. These methods do not search adjacency at all; they encode the constraint into the algebraic object.

Voltage graph lifts continue the same pattern. A small base graph and a finite group define a covering graph whose regularity is inherited from the base. Exoo and Jajcay formalize the relation between voltages on closed walks in the base graph and the girth of the lift [14]. This reduces the search from $O(n^2)$ adjacency choices on a lifted graph to a much smaller voltage assignment on the base.

A second family of methods relies on local search inside the space of k-regular graphs of a fixed order. Hill climbing and tabu search use edge swaps that preserve regularity and minimize a cycle-based cost function; tabu lists prevent the search from immediately reversing recent moves. Excision methods take a known small cage, remove a carefully chosen subgraph (typically a tree or a small structured neighborhood), and reconnect the remaining deficient vertices to obtain a regular graph one to several vertices below the previous best [15], [16]. Constraint programming and integer programming encodings have also been studied, but the size of the target graphs limits their direct applicability.

Recent computational work combines several of these ideas. Exoo et al. couple voltage-lift generation with advanced tabu search, hill climbing using a distance distribution cost, and tree excision, and report eleven new upper bounds on cage orders in a single pipeline [4]. The takeaway for this thesis is methodological: improvements come from chaining structurally constrained search methods, not from any single algorithm.

These methods provide the natural non-learning baselines. A learned method is not very informative if it only beats unrestricted random edge editing. A stronger comparison asks whether learning improves a search process that already uses graph-theoretic structure, such as a voltage-lift search with exact short-cycle checks.

3.2 Graph Neural Networks and Their Limits

The machine-learning side is closest to work on graph neural networks and learned heuristics for combinatorial search, not only to graph generation. Work such as NeuroSAT showed that neural networks can learn useful signals on symbolic search instances even when they do not replace the exact solver [17]. The analogy here is that a learned component may guide which states or moves are explored first, while exact algorithms still verify the final graph.

Standard message-passing GNNs are well-suited to permutation-equivariant scoring of graph states, but their expressivity is bounded by the one-dimensional Weisfeiler–Leman test [6], [9]. Garg, Jegelka, and Jaakkola prove that local message-passing GNNs cannot compute several global graph invariants, including girth and diameter, and give the first data-dependent generalization bounds for such models [18]. This is precisely the regime of the cage problem: high girth, vertex-transitive or nearly so, and locally tree-like. On such graphs, L-layer message-passing GNNs

collapse to nearly identical vertex embeddings, and standard architectures cannot resolve the global structure required to verify or score a candidate construction.

Two responses to this limit are relevant here. Subgraph-based architectures add explicit cycle or motif counts as structural features, which lifts expressivity above plain 1-WL message passing [19]. Cycle-aware hierarchies such as the Loopy framework [12] extend message passing with cycle bases. Both lines of work are used in this thesis as candidate architectures when the prediction target depends on cycle information.

Degree regularity, on the other hand, can be enforced by construction, and short-cycle violations can often be detected exactly. This motivates learned scoring and move ranking rather than replacing the mathematical verification step. Reinforcement learning has also been applied to extremal graph problems; that prior work is discussed in Chapter 5, once the construction formulation that makes the comparison meaningful is in place.

4 Probing GNNs on Graph Properties

Degree prediction and minimum-cycle prediction serve as controlled probes for the two structural constraints that define a (k, g)-graph. In both tasks the graph is given and the model assigns a label to each vertex.

Degree prediction is local and exactly verifiable. A vertex degree can be recovered from the immediate neighborhood, so this task tests whether the model, data pipeline, and training loop can learn a simple structural quantity. If an architecture fails to predict degree reliably, it is not a good candidate for harder construction tasks without further tuning.

Minimum-cycle prediction is a harder structural task. For each vertex, the target is the length of the shortest cycle containing that vertex, with target 0 for vertices that do not lie on any cycle. Unlike degree, this cannot be determined from immediate neighbors alone. It requires information about paths that leave a vertex and later return to it. This makes the task closer to the girth constraint used in constructing (k, g)-graphs.

4.1 Data Generation

Both tasks use dynamically generated random graphs. Instead of training on a fixed set of graphs, new Erdos–Renyi graphs are sampled during training. This reduces the chance that a model simply memorizes a finite training set and gives a cheaper way to expose it to many small graph structures.

Labels are generated by exact graph algorithms. Degree labels are computed as $\deg(v)$ for each vertex. Minimum-cycle labels are computed by searching for the smallest cycle containing each vertex. This keeps the supervision reliable even though the graphs themselves are random.

The input features are intentionally simple. The experiments use either a constant one-dimensional feature or a four-dimensional feature vector consisting of a normalized node index, two random real-valued coordinates, and one additional random scalar. These features should not be interpreted as meaningful graph attributes. They mainly give the neural network a vertex-level input tensor while forcing the model to learn from graph structure.

4.2 Evaluation

The tasks are integer-valued, so a single accuracy number is too coarse: a prediction off by one is qualitatively different from one off by five. Training uses a regression objective with mean squared error, and the results are reported using exact-rounded accuracy together with mean absolute error, root mean squared error, and the fraction of predictions that are off by one.

The evaluation battery contains 75 graphs with 1790 vertices in total. It combines random Erdos–Renyi graphs with structured examples such as complete graphs, complete bipartite graphs, cycles, grids, hypercubes, and known regular graphs such as the Petersen and Heawood graphs. Each trained model is evaluated on the same graphs.

The tables use compact labels. A *small* model has hidden dimension 32 and 4 layers; a *large* model has hidden dimension 128 and 6 layers. A row marked as *best variant* reports the strongest model from that family in the validation run, rather than listing every trained configuration.

4.3 Degree Prediction Results

Each model is evaluated on the same battery of 75 graphs containing 1790 vertices in total. The trained variants are denoted *small* (hidden dimension 32, 4 layers) and *large* (hidden dimension 128, 6 layers); Loopy models additionally use a rank-3 cycle basis with weight sharing across layers. Table 1 lists the metrics.

Model	Accuracy	MAE	RMSE
SAGE, small	1.000	0.000	0.000
GIN, small	1.000	0.000	0.000
SAGE, large	1.000	0.000	0.000
GIN, large	0.998	0.002	0.041
GPS-GIN, small	0.998	0.002	0.041
GPS-GIN, large	0.978	0.022	0.148
Loopy, large	0.401	0.925	2.876
Loopy, small	0.366	0.850	1.182
GCN, small	0.336	0.932	1.261
GCN, large	0.307	1.087	1.449

Table 1: Degree prediction metrics by architecture.

GIN and SAGE variants learn degree essentially perfectly at both capacities. GPS-GIN matches them within rounding at the small setting and degrades slightly when made larger. The degree-normalized GCN variants are the weakest, consistent with the expectation that dividing messages by neighborhood size suppresses raw counting information. Loopy GNN, designed to expose cycle information, performs comparably to GCN here: its extra cycle channel does not help on what is essentially a single-hop counting problem.

4.4 Minimum-Cycle Prediction Results

Minimum-cycle prediction is substantially harder. The target asks for the length of the shortest cycle containing each vertex, with value 0 for vertices not contained in any cycle. This requires information beyond the immediate neighborhood and is closer to the girth constraint that appears in constructing (k, g)-graphs. The evaluation uses the same 75-graph battery and the same *small* / *large* capacity conventions as the degree task. Table 2 lists the metrics.

Model	Accuracy	MAE	RMSE	Off by 1
Loopy, large	0.647	1.287	2.834	0.193
Loopy, small	0.393	1.969	3.162	0.182
GIN, small	0.391	1.346	2.125	0.253
SAGE, large	0.266	1.645	2.432	0.319
GPS-GIN, small	0.253	3.382	4.138	0.000
GCN, large	0.188	1.888	2.441	0.255
GCN, small	0.161	2.015	2.533	0.220

Table 2: Minimum-cycle prediction metrics by architecture.

The larger Loopy model is the clear winner, which supports the intuition that cycle-aware architecture is useful for cycle-related tasks. Even so, the task is not solved: the best result still misses roughly a third of the vertices. This negative result is useful: it suggests that girth-related information should not be treated as a simple supervised prediction target, and it motivates the later move toward search formulations where exact graph-theoretic checks remain part of the algorithm.

4.5 Lessons From the First Experiments

A model can fail because its aggregation rule is poorly matched to the task, because its capacity is too small, or because it lacks useful structural features. The larger validation run separates these effects more clearly: SAGE solves degree even at small capacity, while the larger Loopy model is the only model that performs substantially better than the rest on minimum-cycle prediction.

The conclusion is that the learning formulation matters. Degree is a local counting problem; minimum-cycle prediction is a cycle-sensitive structural problem; constructing a (k, g)-graph is harder still because the model must produce a graph satisfying both constraints, not predict a property of one given.

5 Construction Methods for (k, g)-Graphs

The target problem is to construct small (k, g)-graphs. This is harder than predicting a graph invariant, because the algorithm must actively choose edges while keeping the partial object close to a valid regular graph. The search space grows quickly with the number of vertices, and most arbitrary edge choices lead to invalid or unpromising graphs.

The problem is therefore more naturally viewed as a search problem than as a single prediction problem. A construction algorithm repeatedly holds a partial object, chooses a modification, checks which constraints are still satisfiable, and continues until it either reaches a valid graph or enters an unpromising region of the search space. Machine learning can enter this process in several different ways: as a learned heuristic for deterministic search, as a policy trained from complete construction attempts, or as a scoring function inside a constrained algebraic search.

5.1 Guided Search as the First Idea

A natural first idea is to use deterministic search and let a GNN guide it. For example, one can build a graph edge by edge and use a learned scoring function to prioritize partial graphs that appear more likely to extend to a valid (k, g)-graph.

The difficulty is not the search loop itself. The difficulty is that ordinary supervised training does not apply: there is no straightforward way to label partial states well enough to train a model to predict whether to expand them. Positive examples can be obtained by taking subgraphs of known valid graphs. Useful negative examples are harder: a partial graph may look bad but still be extendable through further

edits. Deciding whether a partial graph can be completed into a valid (k, g)-graph leads back to the same kind of problem. A supervised heuristic can therefore end up training mostly on easy examples far from the decision boundary.

This explains why a purely supervised A-star-style approach was not pursued as the main construction method.

5.2 From Guided Search to Reinforcement Learning

The attractive scenario would be a good heuristic that guides A-star-style search without a pre-built dataset. Reinforcement learning provides exactly this: instead of asking for a dataset that says whether each partial graph is extendable, the model learns from complete construction attempts. The environment supplies rewards based on the observed consequences of actions: whether an edge edit was valid, whether the graph moved closer to regularity, whether short cycles were avoided, and whether the final graph satisfies the target constraints.

In this sense, the reinforcement-learning approach is not separate from search. It is a learned version of heuristic search. A hand-designed search algorithm chooses moves according to a manually specified rule; an RL policy attempts to learn such a rule from repeated interaction with the construction environment.

It is important to keep in mind that direct reinforcement learning does not build the target constraints into the formulation: k-regularity and the girth lower bound are not encoded by the action space or the reward in any exact way. The agent must learn which vertices to activate, which edges to add or remove, how to make every active vertex reach degree k, and how to avoid creating cycles shorter than g. The direct-RL formulation introduced in the next section is therefore useful partly as a diagnostic: it tests how far an unconstrained learned search can go before the constraints have to be enforced explicitly in the representation.

5.3 Direct Reinforcement Learning

In the direct reinforcement-learning formulation, construction is represented as a sequential decision problem and trained with Proximal Policy Optimization [20]. The state is a partially constructed graph. Each action selects an unordered pair of vertices. If the edge is absent, the action attempts to add it; if the edge is present, the action attempts to remove it.

This direct graph-editing formulation has an advantage: it does not require a precomputed dataset of labeled partial graphs. The model learns from interaction. However, it also makes the action space and the reward design essential, and these are addressed in the next subsections.

Several invalid or unhelpful actions are removed from the action space before the policy sees them, so the agent cannot select them at all:

- An edge cannot be added if either endpoint already has degree k.

- An edge cannot be added if it would create a cycle shorter than g.
- An edge cannot be removed if doing so would disconnect the active subgraph.
- An edge cannot be removed if it would leave too few active vertices to reach the Moore-bound target.

5.4 Curriculum Learning

The motivation for the curriculum is direct: at uniform difficulty an untrained agent almost never produces a valid graph, so the learning signal collapses and nothing is learned. Starting from parameter pairs with a small Moore bound raises the per-episode success rate enough that the agent has something to learn from, and the difficulty can then be increased once that signal is reliable.

The environment orders parameter pairs by their Moore bound and unlocks a new pair only after the agent reaches a required success rate over a recent window of episodes. Training starts from $(3, 5)$.

Curriculum learning is what makes the direct RL formulation learn at all. Without it, the agent never produces a valid $(3, 5)$-graph. With the curriculum in place, the agent learned the $(3, 5)$-graph and the $(3, 6)$-graph stages — the rest is addressed by the reward design below.

5.5 Reward Shaping

The environment uses a shaped reward. Earlier reward designs concentrated almost all signal at the end of an episode, and successful episodes were too rare for that signal to guide learning reliably. A per-step progress term was added so that the agent receives a positive signal when an action moves the partial graph closer to a valid construction and a negative one when it moves away. A valid final graph receives a large terminal reward, but the per-step term carries most of the learning signal.

The progress term is implemented as a potential-based shaping signal [21]. Let s denote a state (a partial graph) and let $\Phi(s)$ denote its *potential*: a scalar that grows as the active subgraph approaches a k-regular graph on the Moore-bound number of vertices, rewarding both each active vertex approaching degree k and the total edge count approaching the corresponding $kM/2$ edges. The per-step shaping reward is

$$\Phi(s') - \Phi(s),$$

where s' is the state reached after taking the action in state s. The value is positive when the partial graph moves closer to the target and negative when it moves away.

5.6 Limits of the Direct Formulation

The direct graph-editing formulation is useful as a baseline, but it has structural weaknesses. Regularity is not guaranteed; it must be learned or enforced through action pruning and rewards. The action space also grows quadratically with the

number of available vertices: on a graph with n vertices, there are $\binom{n}{2} = \frac{n(n-1)}{2}$ possible unordered pairs, and each pair can become an add-or-remove decision depending on the graph state. Finally, success on small target pairs does not by itself imply that the method scales to harder open cases.

Its main role in the thesis is methodological: it shows that learning from interaction is possible, but that too much capacity is spent rediscovering constraints that are already known from graph theory. The later experiments therefore compare direct PPO against random search, heuristic search, and voltage-lift methods rather than interpreting it in isolation.

5.7 Comparison with Prior Reinforcement Learning Work

The direct formulation reaches the same conclusion that has been reported in related work on RL for extremal graph problems. Wagner's cross-entropy method succeeds on counterexample tasks where the target graphs have at most a few tens of vertices and the objective is a single scalar conjecture [22]. It does not scale to dense joint constraints such as exact k-regularity combined with high girth. The RLGT framework [23] formalizes the same setting and reports similar fragility under multi-objective structural rewards. Both observations agree with the behavior seen here: unrestricted edge-level RL must spend most of its capacity rediscovering constraints that are already known from graph theory, and the useful learning signal is correspondingly sparse.

Where RL has succeeded for related graph problems, the action space was constrained by structure. Freire et al. apply RL to LDPC components of hypergraph product codes [24]; the learned policy operates on a product construction rather than on arbitrary adjacency, and it outperforms the standard girth-based heuristic. The expressivity limits of standard message-passing GNNs on locally tree-like graphs [18] make this constraint not only convenient but structurally necessary: without it, the policy cannot reliably distinguish promising partial states. These prior results therefore support, rather than contradict, the conclusion drawn from the direct experiments. They motivate the transition to voltage lifts in the next section, where regularity is built into the representation and the learning problem is reduced to choosing voltage assignments on a small base graph.

5.8 Transition to Voltage Lifts

Voltage graph lifts provide a different representation of the construction problem, not a different neural-network architecture. The search starts with a small base graph and a finite group. The algorithm assigns group elements, called voltages, to the base graph edges. The lift then produces a larger graph.

This representation is attractive because regularity is built into the construction: if the base graph is k-regular, the lift is also k-regular. The learning problem can

therefore focus more directly on avoiding short cycles and choosing promising voltage assignments.

Thus the transition from direct RL to voltage lifts is not a change from one arbitrary experiment to another. It is a response to the weaknesses of the first formulation. The direct environment made the construction problem as general as possible; the voltage formulation instead uses known algebraic structure to remove regularity from the learning problem and to provide a cheaper exact signal for girth violations.

This formulation aligns better with recent non-AI work on small regular graphs, where pruning, tabu search, hill climbing, and excision are applied inside highly constrained search spaces [4]. The detailed construction of voltage lifts is described next.

6 Voltage Graph Lifts

Voltage graph lifts replace edge-by-edge construction with a smaller algebraic choice. If the base graph is k-regular, then every lift of that base graph is also k-regular. The search therefore shifts from constructing all edges of a large graph to choosing group labels on the edges of a much smaller base graph.

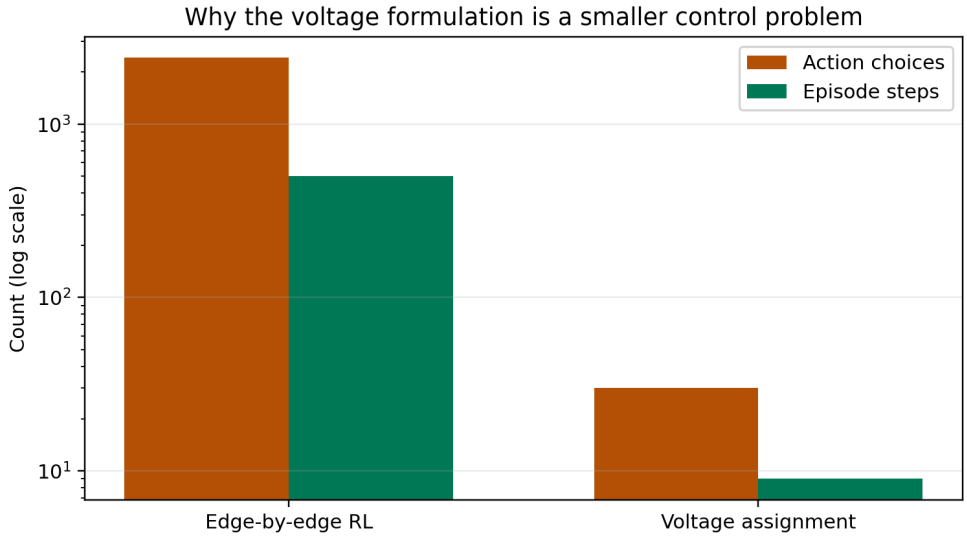

Figure 1: The voltage formulation has a much smaller control problem than direct edge-by-edge graph editing. The numbers shown are illustrative: a 70-vertex direct environment has thousands of possible edge actions, while a voltage assignment step chooses one group element.

6.1 Construction

A voltage-lift construction uses three ingredients:

- a small base graph B,
- a finite group Γ,

- a voltage assignment α that maps every directed edge of B to an element of Γ.

For undirected graphs, each edge is represented by two opposite directed arcs. If one direction has voltage a, then the reverse direction receives a^{-1}. The lift has vertex set $V(B) \times \Gamma$. For an arc from u to v with voltage a, the lift connects

$$(u, h) \quad \text{to} \quad (v, ha)$$

for every group element $h \in \Gamma$. This thesis uses right multiplication by the voltage element. For abelian groups this convention is invisible, but it matters for non-abelian groups because the order of multiplication is part of the construction.

The resulting graph has $|V(B)| \cdot |\Gamma|$ vertices. The verification step checks connectedness, k-regularity, and girth.

The following figure illustrates the construction on the base graph with two vertices and three parallel edges. The group is Z_7, so each base vertex becomes seven vertices in the lift. If a base edge has voltage a, then from each copy (u, h) the lifted edge goes to $(v, h + a)$, where addition is taken modulo 7. The three base edges therefore define three perfect matchings between the two groups of seven lifted vertices. With voltages 0, 1, and 3, the resulting graph has 14 vertices, is 3-regular, and has girth 6.

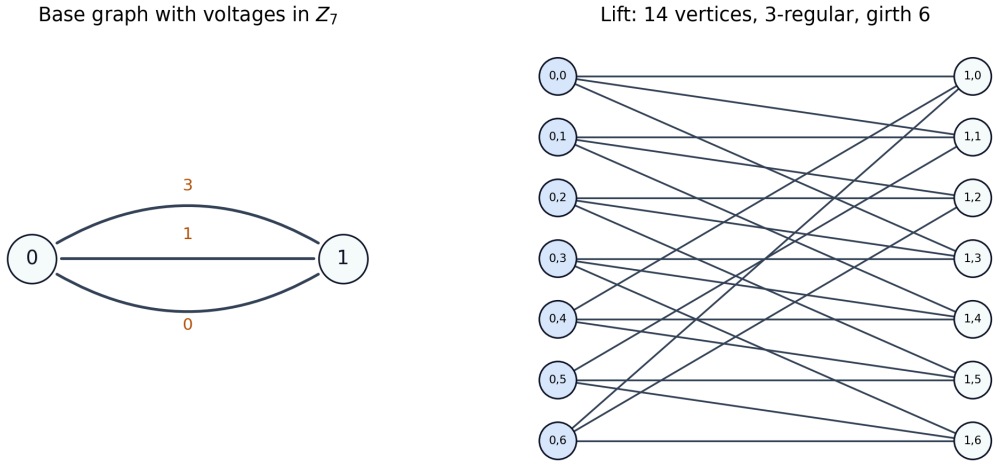

Figure 2: A two-vertex base graph with three parallel edges and voltages in Z_7 lifts to a 14-vertex cubic graph. With a suitable assignment, this produces the Heawood graph, a $(3, 6)$-graph of minimum order.

6.2 Why This Helps

The most important property is preservation of regularity. In direct graph editing, the agent must learn to keep every vertex near degree k. In the voltage formulation, this is structural. A k-regular base graph produces a k-regular lift automatically. This is why the approach is a better match for constructing (k, g)-graphs than unrestricted graph generation.

The second advantage is that girth can be analyzed on the base graph. Consider a closed non-backtracking walk W in the base graph. Multiplying the voltages along the directed arcs of W gives its net voltage. If this product is the identity element of Γ, then following the corresponding lifted edges returns not only to the same base vertex but also to the same group element. Such a walk therefore creates a cycle candidate in the lift. Conversely, every cycle in the lift projects to a closed walk in the base graph with identity net voltage.

This is the reason voltage lifts are useful computationally: short-cycle violations can be detected by enumerating short closed walks in the base graph and multiplying voltages along the walk, without constructing every candidate lift in full.

This gives a cheap exact cost for search:

- if a short closed walk has identity net voltage, it corresponds to a short cycle in the lift;
- if no such walk exists below length g, the lift has girth at least g up to the checked bound.

This exact cost is used later by the search and reinforcement-learning experiments in the experiments chapter. It is also the main reason the learned component can be placed inside a constrained search procedure instead of being responsible for generating an entire graph from scratch.

6.3 Search Methods in the Current Framework

The voltage framework contains several search strategies. They differ in how they choose voltage assignments, but they all keep exact verification at the end: a generated graph is accepted only after connectedness, regularity, and girth are checked.

Method	Search object	Guidance signal	Verification
Exhaustive search	All voltage assignments	None; enumerates all choices	Exact lift check
Random search	Sampled voltage assignments	Uniform random choices	Exact lift check
Tabu search	One-voltage changes	Number of short identity-voltage walks	Exact lift check
GNN-guided beam search	Partial voltage assignments	`GirthPredictor` score	Exact lift check
Meta-search	Base graph and group choices	Enumeration over configured families	Exact lift check
Voltage-assignment PPO	Sequential voltage assignment	Learned policy and reward	Exact lift check

Table 3: Search strategies in the voltage framework. The learned components guide the search, but they do not replace exact verification of the resulting (k, g)-graph.

For small groups and few base edges, exhaustive search enumerates every voltage assignment. This is only practical when $|\Gamma|^m$ is small, where m is the number of undirected base edges. Random search samples assignments and verifies them; it is simple, but it is an important baseline because some small cases are easy enough that random search succeeds.

The tabu search follows recent computational work on small regular graphs [4]. It fixes tree-edge voltages to the identity, so only non-tree edges are searched. It then changes one voltage at a time and minimizes the number of short identity-voltage walks. A tabu list prevents the algorithm from immediately undoing recent moves.

The GNN-guided beam search assigns voltages edge by edge. A trained `GirthPredictor`, defined and evaluated in Chapter 7, scores partial assignments, and the search keeps only the highest-scoring candidates. The `meta_search` function searches over base graphs and finite groups. For cubic graphs, the candidates include the dumbbell base, a four-node cubic base, the prism base, and a Petersen-like base. Candidate groups include cyclic groups, dihedral groups, direct products, and semidirect products.

The reinforcement-learning environment in `ai/cage/voltage/rl_env.py` treats one voltage assignment as one episode. The state is the base graph with partial voltage labels, the action is a group element, and the episode length is the number of base edges. This is much shorter than the direct edge-editing environment. The model in `ai/cage/voltage/rl_model.py` uses either a GINE-only encoder or a GPS encoder

with GINE as the local message-passing component, followed by global pooling, an actor head over group elements, and a critic head for PPO.

6.4 Interpretation

Voltage lifts are not new, and many hand-crafted or computational lift searches already exist. The relevant question for this thesis is whether graph neural networks can guide this constrained search space better than simple baselines.

This comparison makes negative results useful rather than embarrassing. If the learned component does not outperform random search, tabu search, or beam search, then the chosen learned representation did not capture enough reusable structure. If it does outperform them on some configurations, the result is more meaningful because the comparison is against methods that already respect the mathematics of the problem.

7 Experiments and Results

The following experiments test the learned components inside the voltage formulation: a predictor for voltage assignments, reinforcement learning over voltage choices, and a comparison of complete generators for (k, g)-graphs.

7.1 Voltage-Girth Predictor

The voltage-lift experiments use a `GirthPredictor` to score voltage-labeled base graphs. The input is a small directed base graph with node features and edge attributes encoding voltage labels. The model embeds the voltage labels, applies several GINE layers, pools the graph representation, concatenates global context features $(k, g, |\Gamma|)$, and predicts whether the final lift will reach the target condition girth $\geq g$. An auxiliary girth-regression output is trained as an additional signal, but the binary target is the quantity that directly matches the search decision: whether a candidate should be preferred for a given (k, g) target.

The predictor is trained as a single (k, g)-independent model. The target degree and target girth are input features, not separate choices of model weights. This matters methodologically: a useful learned heuristic should reuse information across targets instead of requiring a new model for every pair (k, g).

The model has overall F1 score 0.694, but this aggregate number hides important variation between targets. It performs well on easier or denser-positive targets such as $(3, 5)$, $(3, 6)$, $(4, 5)$, and $(4, 6)$, but it weakens sharply on sparse targets such as $(4, 7)$ and $(4, 8)$. The target $(5, 7)$ has no positive examples in the generated dataset, so its F1 score is 0 even though accuracy is 1.000; this is exactly why accuracy alone is misleading for this task.

Target	F1	Target	F1	Target	F1
$(3,5)$	0.823	$(4,5)$	0.779	$(5,5)$	0.613
$(3,6)$	0.800	$(4,6)$	0.800	$(5,6)$	0.613
$(3,7)$	0.659	$(4,7)$	0.147	$(5,7)$	0.000
$(3,8)$	0.524	$(4,8)$	0.248		
$(3,9)$	0.488				
$(3,10)$	0.370				

Table 4: Per-target F1 scores for the voltage-girth predictor. The aggregate score is not enough; the hard sparse-positive targets are visibly weaker.

The predictor learns a nontrivial signal about voltage assignments, especially for smaller targets, but it is not uniformly reliable across the search space. This supports using it only as a heuristic inside verified search, not as a replacement for exact girth checking.

7.2 Voltage-Assignment Reinforcement Learning

The voltage formulation was also used as a reinforcement-learning environment. Each episode assigns voltages to the base graph edges. This is much shorter than direct edge editing: the policy chooses group elements for a small base graph rather than choosing among all possible edges of a large graph.

Two GPS-based voltage policies were trained. Both used three message-passing layers and PPO. The smaller model used hidden dimension 64 and four attention heads; the larger model used hidden dimension 128 and eight heads. Both runs reached stage 6 of the curriculum after 6.5 million environment steps.

Voltage PPO model	Hidden dim.	Heads	Final steps	Final stage
Compact GPS policy	64	4	6.5M	6
Larger GPS policy	128	8	6.5M	6

Table 5: Voltage-assignment PPO training progress. Both policies unlocked six curriculum stages, showing that the voltage environment is learnable, but not yet proving superiority over non-learned search.

The stage-unlock histories show a useful difference between the two runs. The larger model reached intermediate stages faster: it unlocked stage 4 after roughly 32 thousand steps and stage 5 after roughly 469 thousand steps, while the compact model reached the same stages after roughly 197 thousand and 1.30 million steps. However, both runs required several million steps to reach stage 6, and the best

average reward became worse on later stages. This indicates that the curriculum becomes substantially harder and that reaching early stages is not enough evidence of scalable construction of (k, g)-graphs.

7.3 Generator Comparison

The central question is whether learning improves search for (k, g)-graphs. The predictor and the voltage-RL policies show that the learned components are meaningful, but they must be compared against structured non-learned baselines. The generator comparison evaluates 13 target pairs, from $(3, 5)$ to $(5, 7)$, with two random seeds per method. For voltage-RL inference, actions are sampled from the trained policy distribution. This avoids collapsing the stochastic policy to a narrow set of voltage choices.

Method	Success	Med. time	Med. steps	Med. order
Random edge editing	10/26	1.5 s	1221	26
Heuristic search	12/26	1.1 s	53	26
Direct PPO	4/26	17.2 s	2832	19
Voltage search	20/26	3.1 s	47	60
Voltage + predictor	20/26	5.3 s	56	60
Voltage PPO, h128	20/26	0.54 s	48	50
Voltage PPO, h64	20/26	0.53 s	112	50

Table 6: Aggregate generator comparison. Medians are computed only over successful trials. Step counts are not directly comparable across method families, because they count different operations.

The most important result is that the voltage representation changes the difficulty of the problem. Direct random search and heuristic search solve some small targets, but they fail more often as g or k increases. Direct PPO succeeds on $(3, 5)$ and $(4, 5)$, but it remains much weaker than the voltage methods. Plain voltage search, the predictor-guided version, and both voltage-RL policies all succeed on 20 of 26 trials. This supports the main structural claim of the thesis: building regularity into the representation is more valuable than asking a neural policy to rediscover it from edge edits.

Target	Rand.	Heur.	PPO	Volt.	Volt. +P	VRL
$(3,5)$	2/2	2/2	2/2	2/2	2/2	4/4
$(3,6)$	2/2	2/2	0/2	2/2	2/2	4/4
$(3,7)$	0/2	2/2	0/2	2/2	2/2	4/4
$(3,8)$	2/2	2/2	0/2	2/2	2/2	4/4
$(3,9)$	0/2	0/2	0/2	2/2	2/2	4/4
$(3,10)$	0/2	0/2	0/2	2/2	2/2	4/4
$(4,5)$	0/2	0/2	2/2	2/2	2/2	4/4
$(4,6)$	2/2	2/2	0/2	2/2	2/2	4/4
$(4,7)$	0/2	0/2	0/2	0/2	0/2	0/4
$(4,8)$	0/2	0/2	0/2	0/2	0/2	0/4
$(5,5)$	0/2	0/2	0/2	2/2	2/2	4/4
$(5,6)$	2/2	2/2	0/2	2/2	2/2	4/4
$(5,7)$	0/2	0/2	0/2	0/2	0/2	0/4

Table 7: Success counts by target. `Volt.+P` uses the voltage-girth predictor, and VRL combines both voltage-RL policies.

The success matrix alone would suggest a tie between the strongest voltage methods, because plain voltage search, the predictor-guided search, and both voltage-RL policies solve the same ten target pairs. The difference appears in the cost of the search. On several harder successful targets, the voltage-RL policies find valid lifts much faster than unguided voltage search.

Target	Voltage	Volt. +P	VRL128	VRL64
$(3,7)$	1.55 s	5.30 s	0.53 s	0.53 s
$(3,9)$	10.16 s	9.32 s	1.62 s	3.07 s
$(3,10)$	36.79 s	37.81 s	2.57 s	16.81 s
$(5,5)$	12.19 s	8.30 s	6.07 s	10.11 s
$(5,6)$	24.53 s	13.71 s	1.55 s	1.54 s

Table 8: Median successful wall-clock time on selected nontrivial targets. The voltage-RL policies do not solve more targets, but on several targets they reach a valid lift faster.

This is the clearest positive result for machine learning in the construction experiments. The learned policy does not expand the set of solved targets, but it can reduce the time needed to find a valid voltage assignment. In this role, the model acts as a heuristic inside a constrained and exactly verified search space.

There is also an important tradeoff. The voltage-RL policies often reach valid graphs by using larger lift orders than plain voltage search. For example, on $(3, 9)$ plain voltage search finds graphs of order 60, while the voltage-RL policies find graphs around order 90. On $(3, 10)$ plain voltage search finds order 108, while the voltage-RL policies may use orders above 170. The reward therefore optimizes feasibility and speed, not minimality. They are therefore generators of valid (k, g)-graphs rather than cage-finding methods.

The learned girth predictor gives weaker evidence. It ties plain voltage search in success count and is slower in the aggregate median. It helps on some targets, such as $(5, 6)$, but the overhead of scoring partial assignments can outweigh the benefit when unguided voltage search is already cheap. The hard targets $(4, 7)$, $(4, 8)$, and $(5, 7)$ remain unsolved by every tested method, and they are also the targets where the predictor has weak or zero F1 scores.

Overall, the generator comparison supports a restrained conclusion. The experiments do not show that machine learning already solves harder target pairs than structured non-learned voltage search. They do show that learned voltage policies can reduce search time on successful targets, while exact verification and the voltage representation remain responsible for the mathematical correctness of the output.

8 Future Work

The experiments suggest that future progress should come from combining learned components with constrained graph-theoretic search. Direct edge editing is too unstructured: the agent has to learn regularity, avoid short cycles, choose useful vertices, and recover from bad intermediate states at the same time. Voltage lifts are a more promising representation because regularity is built in and short-cycle violations can be detected through identity-voltage walks in the base graph.

The most direct next step is stronger voltage-lift search before learning is applied. Recent computational work on small regular graphs uses constraints from short closed walks, spanning-tree normalization, canonicalization under automorphisms, and rejection of equivalent assignments [4]. Such pruning would reduce the search space that the learned component has to guide. It would also make the comparison with non-learned baselines stronger, because the learned method would be tested inside a more mature version of the same mathematical search framework.

The learned models could also receive better training signals. The short-walk cost used by tabu search is a dense signal: it counts how many short closed walks have identity net voltage. This quantity could be used as an auxiliary prediction target, as reward shaping, or as part of a beam-search score. Hard targets also need better data. More balanced sampling near the decision boundary, more diverse base graphs and groups, and explicit hard-negative mining could make the scorer more reliable where positive examples are rare.

The voltage-RL results suggest a particularly concrete modification. The learned policies can find valid lifts quickly, but sometimes by choosing larger lift orders than plain voltage search. A future reward should therefore penalize unnecessary group order or normalize success by the size of the resulting lift. That would align the learned objective more closely with the cage motivation, where smaller valid graphs are more valuable than larger ones found quickly.

A second constrained direction is Cayley graph search. In a Cayley graph, regularity is automatic once a group and a symmetric generating set are chosen, and girth can be checked from the group structure rather than from an arbitrary edge set. This makes Cayley search a strong non-learned baseline: an initial classical search already recovered exact known orders for several small targets, including $(3, 6)$, $(4, 6)$, and $(5, 6)$. The learned role that appears most plausible here is not replacing girth evaluation, which is already cheap for Cayley graphs, but predicting which groups are promising enough to search. A group-level predictor can then prune the catalogue before random and tabu search are run. [Insert final comparison of classical Cayley search and group-guided Cayley search once the pending run finishes.]

Finally, learning need not generate a whole voltage assignment by itself. A more robust role may be move ranking inside an existing search algorithm: score candidate voltage changes in tabu search, select which base graph and group pairs deserve more compute, or choose which partial assignments survive in a beam. Excision gives another natural setting for this idea. Starting from a known regular graph, one removes a local structure and reconnects the remaining vertices while preserving regularity and girth [4], [16]. A learned model could rank which vertices to remove or which reconnection patterns to try first, while exact verification remains unchanged.

9 Conclusion

This thesis investigated whether graph neural networks can help generate graphs of prescribed degree and girth. The work began with preparatory prediction tasks, continued with direct reinforcement learning over graph edits, and then moved to voltage graph lifts as a more constrained algebraic representation of the construction problem.

The preparatory experiments showed a clear separation between local and cycle-related graph properties. Degree prediction was solved almost perfectly by several GIN- and SAGE-based architectures. Minimum-cycle prediction was much harder, and the best result came from a larger cycle-aware Loopy model. This supports the main methodological lesson: GNNs can learn useful graph structure, but girth-related information should not be treated as an easy generic prediction task.

Direct reinforcement learning was a natural first construction method because the problem is sequential and reliable labels for partial graphs are hard to obtain. However, the direct graph-editing formulation forced the agent to learn too many

known constraints at once: regularity, connectivity, edge validity, and avoidance of short cycles. This made it useful as an exploratory baseline, but not as the most promising final formulation.

The voltage-lift framework is the strongest direction developed in the thesis. It builds regularity into the representation and allows short-cycle violations to be evaluated through net voltages on the base graph. The trained voltage-girth predictor learned a nontrivial but uneven signal, and voltage-assignment PPO was able to progress through several curriculum stages and generate valid graphs in the generator comparison.

The generator comparison confirms that this change of representation is the main positive result. Direct reinforcement learning over edge edits solved only a small number of target pairs, while voltage search solved most of the tested targets. The learned voltage policies did not solve more target pairs than the non-learned voltage baseline, but they often found valid voltage assignments faster on the harder successful targets. This is the strongest evidence in the thesis that machine learning can be useful as a search heuristic.

At the same time, the result remains limited. The learned voltage policies often pay for speed by producing larger lifts, so the experiments demonstrate fast feasible construction rather than minimal-order cage construction. The learned girth predictor also does not clearly improve the voltage search in aggregate: it learns a real signal, but the scoring overhead and weak performance on sparse-positive hard targets limit its usefulness.

The main conclusion should remain cautious. Machine learning appears most useful here as a heuristic inside constrained graph-theoretic search, not as an unrestricted graph generator. The clearest outcome of the work is the movement from unrestricted edge editing toward mathematically structured search spaces.

Bibliography

[1] G. Exoo and R. Jajcay, "Dynamic Cage Survey," *The Electronic Journal of Combinatorics*, 2013.

[2] R. M. Tanner, "A Recursive Approach to Low Complexity Codes," *IEEE Transactions on Information Theory*, vol. 27, no. 5, pp. 533–547, 1981, doi: 10.1109/TIT.1981.1056404.

[3] M. Besta and T. Hoefler, "Slim Fly: A Cost Effective Low-Diameter Network Topology," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2014.

[4] G. Exoo, J. Goedgebeur, J. Jooken, L. Stubbe, and T. Van den Eede, "New small regular graphs of given girth: the cage problem and beyond." 2025.

[5] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, 2017.

[6] M. Grohe, "The Logic of Graph Neural Networks." 2022.

[7] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *International Conference on Learning Representations*, 2017.

[8] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems*, 2017.

[9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," in *International Conference on Learning Representations*, 2019.

[10] W. Hu *et al.*, "Strategies for Pre-training Graph Neural Networks," in *International Conference on Learning Representations*, 2020.

[11] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a General, Powerful, Scalable Graph Transformer," in *Advances in Neural Information Processing Systems*, 2022.

[12] R. Paolino, C. Vignac, and A. Loukas, "Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning," in *International Conference on Learning Representations*, 2024.

[13] A. Lubotzky, R. Phillips, and P. Sarnak, "Ramanujan graphs," *Combinatorica*, vol. 8, no. 3, pp. 261–277, 1988, doi: 10.1007/BF02126799.

[14] G. Exoo and R. Jajcay, "On the girth of voltage graph lifts," *European Journal of Combinatorics*, vol. 32, no. 4, pp. 554–562, 2011, doi: 10.1016/j.ejc.2010.11.011.

[15] G. Araujo-Pardo, C. Balbuena, and J. C. Valenzuela, "Constructions of bi-regular cages," *Discrete Mathematics*, vol. 309, no. 14, pp. 4844–4853, 2009, doi: 10.1016/j.disc.2008.04.011.

[16] G. Exoo, "On decreasing the orders of (k, g)-graphs", *Journal of Combinatorial Optimization*, 2023, doi: 10.1007/s10878-023-01092-9.

[17] D. Selsam, M. Lamm, B. Bünz, P. Liang, L. de Moura, and D. L. Dill, "Learning a SAT Solver from Single-Bit Supervision," in *International Conference on Learning Representations*, 2019.

[18] V. K. Garg, S. Jegelka, and T. Jaakkola, "Generalization and Representational Limits of Graph Neural Networks," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[19] G. Bouritsas, F. Frasca, S. Zafeiriou, and M. M. Bronstein, "Improving Graph Neural Network Expressivity via Subgraph Isomorphism Counting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[21] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999.

[22] A. Z. Wagner, "Constructions in combinatorics via neural networks." 2021.

[23] I. Damnjanović, U. Milivojević, I. Đorđević, and D. Stevanović, "RLGT: A reinforcement learning framework for extremal graph theory." 2026.

[24] B. C. A. Freire, N. Delfosse, and A. Leverrier, "Optimizing hypergraph product codes with random walks, simulated annealing and reinforcement learning." 2025.